Runtime Analysis

Vector

Pseudocode Line / Operation	Cost	Executions
OPEN file using filename	1	1
IF file does not open	1	1
PRINT error/RETURN if needed	1	1
WHILE there is another line	1	n
READ line	1	n
SPLIT line into tokens	1	n
IF number of tokens is less than 2	1	n
PRINT error/RETURN if needed	1	n
CREATE new Course	1	n
SET course.courseNumber	1	n
SET course.courseName	1	n
FOR each prerequisite token	1	Up to n^2
ADD token to prerequisites	1	Up to n^2
ADD course to vector	1	n
CLOSE file	1	1
FOR each course in courses	1	n
FOR each prerequisite	1	Up to n^2
SET prerequisiteFound = false	1	Up to n^2
FOR each possibleCourse in courses	1	Up to n^3
IF course number equals prerequisite	1	Up to n^3
SET prerequisiteFound = true	1	Up to n^3
IF prerequisiteFound equals false	1	Up to n^2
PRINT error/RETURN if needed	1	Up to n^2

Worst-Case Runtime: $O(n^3)$

Hash Table

Pseudocode Line / Operation	Cost	Executions
OPEN file using filename	1	1
IF file does not open	1	1
PRINT error/RETURN if needed	1	1
WHILE there is another line	1	n
Read line	1	n
SPLIT line into tokens	1	n
IF number of tokens is less than 2	1	n
PRINT error/RETURN if needed	1	n
CREATE new Course	1	n
SET course.courseNumber	1	n
SET course.courseName	1	n
FOR each prerequisite token	1	Up to n^2
ADD token to prerequisites	1	Up to n^2
INSERT course into hash table	1	n
CLOSE file	1	1
FOR each course in hash table	1	n
FOR each prerequisite	1	Up to n^2
Check hash table for prerequisite key	1	Up to n^2
IF prerequisite does not exist	1	Up to n^2
PRINT prerequisite error/RETURN if needed	1	Up to n^2
PRINT successful load message	1	1

Worst-Case Runtime: $O(n^2)$

Binary Search Tree

Pseudocode Line / Operation	Cost	Executions
OPEN file using filename	1	1
IF file does not open	1	1
PRINT error/RETURN if needed	1	1
WHILE there is another line	1	n
Read line	1	n
SPLIT line into tokens	1	n
IF number of tokens is less than 2	1	n
PRINT error/RETURN if needed	1	n
CREATE new Course	1	n
SET course.courseNumber	1	n
SET course.courseName	1	n
FOR each prerequisite token	1	Up to n^2
ADD token to prerequisites	1	Up to n^2
INSERT course into tree	n	n
CLOSE file	1	1
FOR each course in tree	1	n
FOR each prerequisite	1	Up to n^2
SEARCH tree for prerequisite	n	Up to n^2
IF prerequisite is not found	1	Up to n^2
PRINT prerequisite error	1	Up to n^2
RETURN if prerequisite is invalid	1	Up to n^2
PRINT successful load message	1	1

Worst-Case Runtime: $O(n^3)$

Data Structure Evaluation

Vector

The vector is straightforward to work with and makes it easy to store and access course data. It can also be sorted by course number to produce the alphanumeric course list required by the advising program. One disadvantage is that searching for a specific course may require checking courses one at a time. This becomes less efficient as more courses are added. The

prerequisite validation in this design also requires searching through the vector, contributing to a worst-case runtime of $O(n^3)$. For memory, a vector is fairly efficient, although it may use some extra space as it grows.

Hash Table

The hash table provides efficient access to course information because each course is stored using its course number as a key. This also makes checking for prerequisites more efficient because the program can look for a specific key instead of searching through every course. One disadvantage is that a hash table does not automatically keep the courses in alphanumeric order. To print the required course list, the courses must first be placed into a sortable structure and then sorted. In this design, the hash table has a worst-case runtime of $O(n^2)$ for loading and validating the course data. For memory, a hash table may use some extra space to store the course data and the keys used to access it.

Binary Search Tree

The binary search tree keeps course data organized by course number, which makes it useful for producing the required alphanumeric course list. An in-order traversal can print the courses from lowest to highest without requiring a separate

sorting step. A disadvantage is that performance depends on the shape of the tree. If the tree becomes unbalanced, searching for courses or prerequisites may require moving through many nodes. In the worst case, this can contribute to the $O(n^3)$ runtime identified in this design. For memory, the tree uses some extra space because each course also needs connections to other nodes in the tree.

Recommendation

Based on the requirements of the advising program, I would use the hash table for the final implementation. The hash table provides efficient access to individual courses by using the course number as a key, which works well when an advisor needs to look up a specific course and its prerequisites. It also had the lowest worst-case runtime in my analysis at $O(n^2)$, compared with $O(n^3)$ for the vector and binary search tree. Although the courses must be sorted separately before printing the full alphanumeric list, the faster course lookup and prerequisite validation make the hash table the best overall choice for this program.